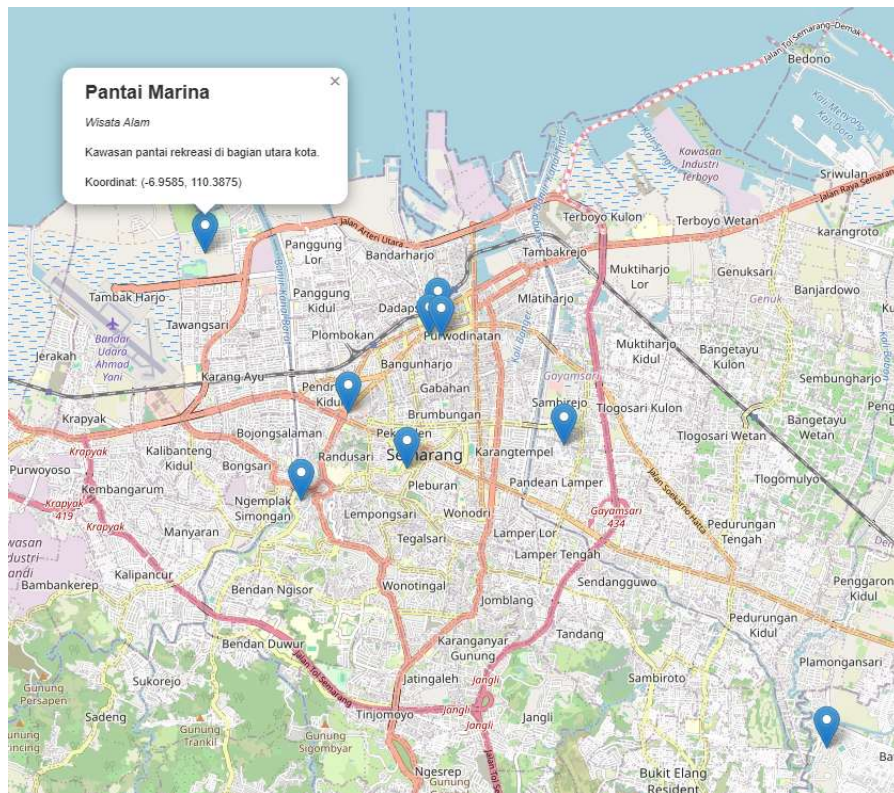




POLITEKNIK NEGERI SEMARANG
JURUSAN TEKNIK ELEKTRO
PROGRAM STUDI STR TEKNOLOGI REKAYASA KOMPUTER

JOBSHEET 12:
INTEGRASI OOP DALAM MINI PROJECT SISTEM
INFORMASI GEOGRAFIS (SIG)



Dosen:

Ir. Prayitno, S.ST., M.T., Ph.D.

Nama Mahasiswa : _____
NIM Mahasiswa : _____

 **Tahun Akademik 2025**

A. Tujuan Instruksional Khusus

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Merancang dan mengimplementasikan struktur kelas menggunakan konsep OOP (kelas, objek, inheritance, polymorphism) untuk merepresentasikan data geografis.
2. Membaca data tabular dari file CSV menggunakan library Pandas.
3. Mengiterasi data dari DataFrame Pandas untuk membuat instance objek.
4. Menggunakan library Folium untuk membuat visualisasi peta interaktif berdasarkan data objek.
5. Menulis data sederhana ke file teks menggunakan mode penulisan ('w') dan penambahan ('a').
6. Mengintegrasikan berbagai konsep pemrograman Python (OOP, file handling, library eksternal) dalam sebuah mini project SIG.

B. Alat dan Bahan

• Perangkat Lunak:

- Sistem operasi Windows/Linux/macOS.
- Python 3.x (versi terbaru yang stabil).
- IDE atau Code Editor (misalnya, PyCharm, VS Code, Jupyter Notebook/Lab, Google Colab).
- Library Python: `pandas` dan `folium`.
 - (Instalasi jika belum ada: `pip install pandas folium`)

• Perangkat Keras:

- Komputer/Laptop dengan spesifikasi memadai.
- Koneksi internet (untuk mengunduh library dan menampilkan peta Folium).

• Bahan Pendukung:

- Jobsheet-jobsheet sebelumnya (OOP Concepts, File Handling).
- Jobsheet ini.
- Contoh file CSV (akan dibuat dalam praktikum).
- Dokumentasi Pandas (<https://pandas.pydata.org/docs/>)
- Dokumentasi Folium (<https://python-visualization.github.io/folium/>)

C. Dasar Teori

Project Sistem Informasi Geografis (SIG) seringkali melibatkan pengelolaan data spasial (lokasi) beserta atribut-atributnya. Pemrograman Berorientasi Objek (OOP) menyediakan cara yang sangat baik untuk menstrukturkan data dan fungsionalitas terkait lokasi ini. Kita dapat mengintegrasikan konsep OOP dengan library Python populer seperti Pandas (untuk manipulasi data) dan Folium (untuk visualisasi peta).

1. Integrasi OOP dalam SIG

- **Kelas untuk Data Geografis:** Kita bisa mendefinisikan kelas dasar (misal: `Lokasi`) yang menyimpan atribut umum seperti nama, latitude, dan longitude. Kelas turunan (misal: `TempatWisata`, `Restoran`, `Kantor`) dapat mewarisi dari `Lokasi` dan menambahkan atribut atau metode spesifik (inheritance).
- **Polimorfisme:** Objek dari kelas turunan yang berbeda dapat diperlakukan secara seragam melalui interface kelas induknya. Misalnya, semua objek turunan `Lokasi` mungkin memiliki metode `get_koordinat()` atau `tampilkan_info_marker()`, tetapi implementasinya bisa berbeda (polymorphism).

- **Enkapsulasi:** Menyembunyikan detail internal representasi data lokasi dan menyediakan metode terkontrol untuk mengakses atau memodifikasinya.

2. File Handling (Mode 'r', 'w', 'a') - Review Singkat

Penanganan file teks dasar tetap relevan, misalnya untuk menyimpan log atau output sederhana.

- **open(nama_file, mode):** Fungsi utama untuk membuka file.
- **Mode:**
 - 'r': Read (Baca) - Membuka file untuk dibaca (default). Error jika file tidak ada.
 - 'w': Write (Tulis) - Membuka file untuk ditulis. **Membuat file baru jika tidak ada, atau MENGHAPUS ISI file lama jika sudah ada.**
 - 'a': Append (Tambah) - Membuka file untuk ditambahkan di akhir. Membuat file baru jika tidak ada. Tidak menghapus isi lama.
- **with open(...) as f::** Konteks manager yang direkomendasikan karena otomatis menutup file setelah selesai atau jika terjadi error.
- **Operasi:** `f.read()`, `f.readline()`, `f.readlines()`, `for line in f:`, `f.write(string)`, `f.writelines(list_of_strings)`.

3. Pandas untuk Membaca CSV

Pandas adalah library fundamental untuk analisis dan manipulasi data di Python. Struktur data utamanya adalah **DataFrame**, yang mirip tabel dua dimensi (baris dan kolom).

- **Membaca CSV:** Fungsi `pd.read_csv()` sangat umum digunakan.

Python

```
import pandas as pd
# Membaca file CSV, asumsi kolom dipisah koma dan ada header
try:
    df = pd.read_csv('nama_file.csv')
    # Parameter umum:
    # sep=', ' (atau ';' , '\t') -> pemisah kolom
    # header=0 -> baris mana yg jadi header (default 0)
    # names=['col1', 'col2'] -> beri nama kolom jika tidak ada header
    print(df.head()) # Tampilkan 5 baris pertama
except FileNotFoundError:
    print("Error: File CSV tidak ditemukan.")
except Exception as e:
    print(f"Error saat membaca CSV: {e}")
```

- **Mengakses Data:**
 - Kolom: `df['nama_kolom']` atau `df.nama_kolom`.
 - Baris berdasarkan indeks: `df.iloc[index]`.
 - Iterasi baris: `for index, row in df.iterrows():` row akan menjadi objek Series
Pandas yang berisi data satu baris.

4. Folium untuk Visualisasi Peta

Folium memudahkan pembuatan peta interaktif berbasis Leaflet.js langsung dari Python. Peta yang dihasilkan berupa file HTML.

- **Membuat Peta Dasar:**

Python

```
import folium
# Tentukan koordinat tengah dan level zoom awal
# Contoh: Semarang
peta_semarang = folium.Map(location=[-6.9929, 110.4200], zoom_start=13)
```

- **Menambahkan Marker:** Menandai titik tertentu di peta.

Python

```
# Tambahkan marker untuk suatu lokasi
folium.Marker(
    location=[-6.9829, 110.4259], # Koordinat (latitude, longitude)
    popup='<i>Simpang Lima</i>', # Konten HTML saat marker diklik
    tooltip='Klik untuk info!' # Teks saat mouse hover
).add_to(peta_semarang) # Tambahkan marker ke objek peta
```

Ada berbagai jenis marker dan ikon yang bisa digunakan (folium.CircleMarker, folium.Icon, dll).

- **Menyimpan Peta:**

Python

```
# Simpan peta ke file HTML
peta_semarang.save("peta_semarang.html")
print("Peta berhasil disimpan ke peta_semarang.html")
```

File HTML ini dapat dibuka di browser web.

Dengan menggabungkan OOP, Pandas, Folium, dan file handling, kita dapat membangun aplikasi SIG mini yang terstruktur dan fungsional.

D. Langkah Praktikum

Praktikum 01: Persiapan Data (Membuat File CSV)

Tujuan: Menyiapkan data sumber dalam format Comma Separated Values (CSV) yang akan digunakan sebagai input pada langkah-langkah praktikum selanjutnya.

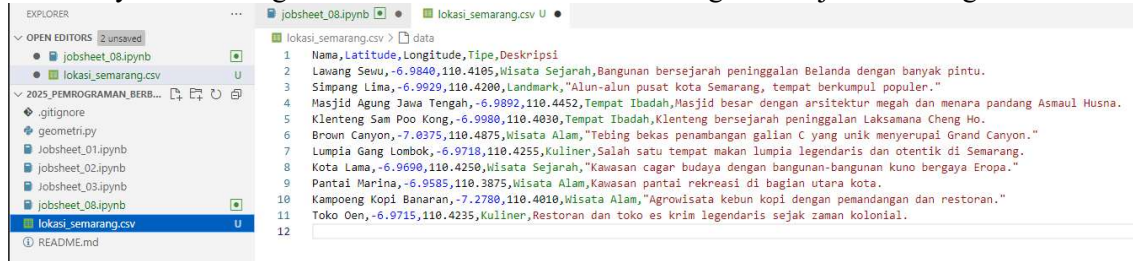
- **Langkah:**

1. **Buka Editor Teks:** Gunakan editor teks apa saja (Notepad, TextEdit, VS Code, Sublime Text, dll.).
2. **Buat File Baru:** Buat sebuah file baru.
3. **Masukkan Data:** Ketik atau salin data berikut ke dalam file. Pastikan setiap nilai dalam satu baris dipisahkan oleh koma (,), dan baris pertama berfungsi sebagai header (nama kolom).

Code snippet

```
Nama, Latitude, Longitude, Tipe, Deskripsi
Lawang Sewu, -6.9840, 110.4105, Wisata Sejarah, Bangunan bersejarah peninggalan Belanda dengan banyak pintu.
Simpang Lima, -6.9929, 110.4200, Landmark, "Alun-alun pusat kota Semarang, tempat berkumpul populer."
Masjid Agung Jawa Tengah, -6.9892, 110.4452, Tempat Ibadah, Masjid besar dengan arsitektur megah dan menara pandang Asmaul Husna.
Klenteng Sam Poo Kong, -6.9980, 110.4030, Tempat Ibadah, Klenteng bersejarah peninggalan Laksamana Cheng Ho.
Brown Canyon, -7.0375, 110.4875, Wisata Alam, "Tebing bekas penambangan galian C yang unik menyerupai Grand Canyon."
Lumpia Gang Lombok, -6.9718, 110.4255, Kuliner, Salah satu tempat makan lumpia legendaris dan otentik di Semarang.
Kota Lama, -6.9690, 110.4250, Wisata Sejarah, "Kawasan cagar budaya dengan bangunan-bangunan kuno bergaya Eropa."
Pantai Marina, -6.9585, 110.3875, Wisata Alam, Kawasan pantai rekreasi di bagian utara kota.
Kampoeng Kopi Banaran, -7.2780, 110.4010, Wisata Alam, "Agrowisata kebun kopi dengan pemandangan dan restoran."
Toko Oen, -6.9715, 110.4235, Kuliner, Restoran dan toko es krim legendaris sejak zaman kolonial.
```

4. **Simpan File:** Simpan file tersebut dengan nama `lokasi_semarang.csv`. Pastikan Anda menyimpannya di direktori kerja yang sama tempat Anda akan menjalankan skrip Python nanti agar mudah ditemukan. Pilih encoding UTF-8 jika memungkinkan.



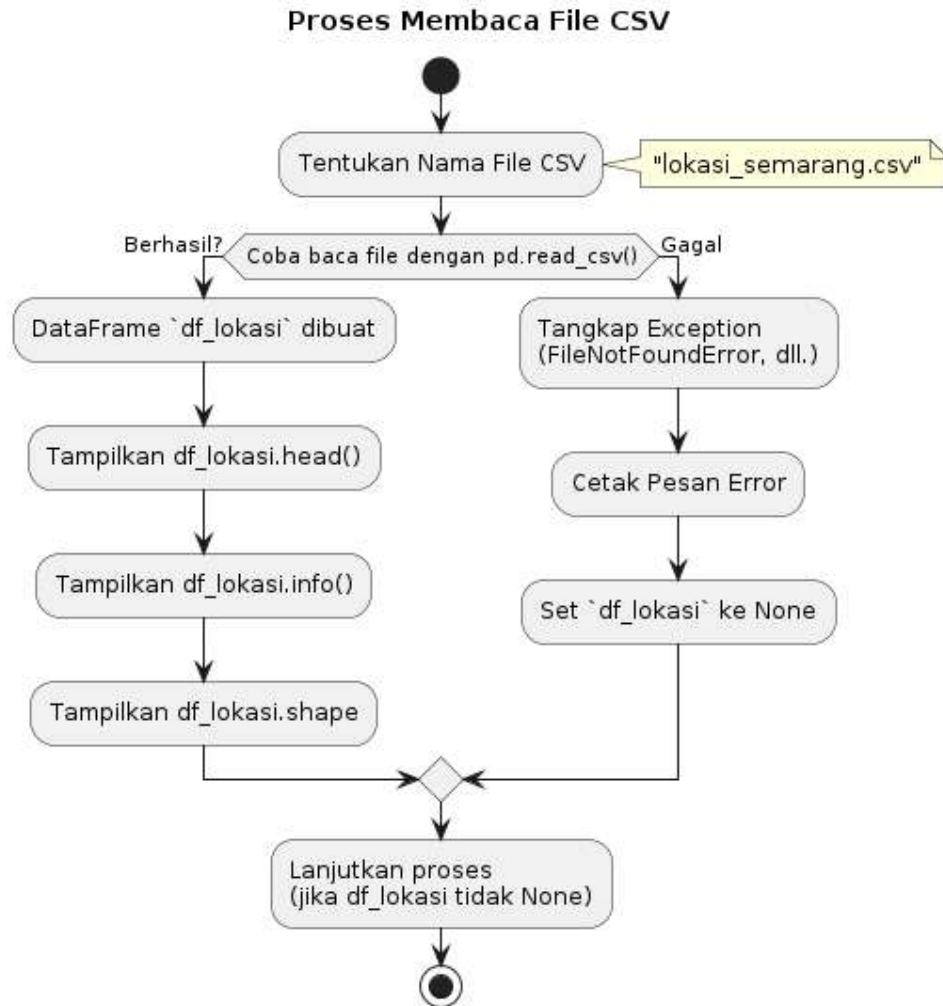
Lokasi_semarang.csv

Observasi:

- Buka file `lokasi_semarang.csv` yang baru Anda simpan menggunakan editor teks atau program spreadsheet (seperti Excel atau Google Sheets) untuk memastikan formatnya benar.
- Periksa apakah ada 6 kolom: Nama, Latitude, Longitude, Tipe, Deskripsi.
- Pastikan pemisah antar kolom adalah koma.
- File ini akan menjadi sumber data utama untuk praktikum selanjutnya.

Praktikum 02: Membaca Data CSV dengan Pandas

- **Tujuan:** Menggunakan library Pandas (`pd`) untuk membaca file `lokasi_semarang.csv` yang telah dibuat pada Praktikum 1 dan memuat datanya ke dalam struktur data `DataFrame` Pandas untuk inspeksi awal.
- **Prasyarat:** Pastikan library Pandas sudah terinstal (`pip install pandas`). File `lokasi_semarang.csv` dari Praktikum 1 harus ada di direktori yang sama.



Penjelasan Kelas Diagram (Diagram Aktivitas):

- **Diagram Aktivitas:** Menggambarkan alur langkah-langkah yang dilakukan dalam kode Praktikum 2.
- **Mulai:** Proses dimulai.
- **Tentukan Nama File:** Menetapkan nama file CSV yang akan dibaca.
- **Coba Baca File:** Merepresentasikan blok `try` yang mencoba memanggil `pd.read_csv()`.
- **Percabangan (if):** Menunjukkan dua kemungkinan hasil dari `try`:
 - **Berhasil:** `DataFrame` berhasil dibuat, lalu dilakukan inspeksi (`head()`, `info()`, `shape`).
 - **Gagal:** Blok `except` dijalankan, pesan error dicetak, dan variabel `DataFrame` diatur ke `None`.

- **Lanjutkan Proses:** Menunjukkan bahwa langkah selanjutnya bergantung pada apakah DataFrame berhasil dibuat.
- **Stop:** Proses berakhir.

Diagram ini fokus pada alur logika pembacaan file CSV menggunakan Pandas, termasuk penanganan error dasar. Ia tidak menggambarkan struktur kelas karena fokus praktikum ini adalah pada penggunaan library eksternal untuk memuat data.

Kode Python:

```
01: # Impor library pandas
02: import pandas as pd
03:
04: # Nama file CSV yang akan dibaca
05: NAMA_FILE_CSV = "lokasi_semarang.csv"
06:
07: def baca_data_lokasi(nama_file: str) -> pd.DataFrame | None:
08:     """
09:     Membaca data lokasi dari file CSV menggunakan Pandas.
10:
11:     Args:
12:         nama_file (str): Path atau nama file CSV yang akan dibaca.
13:
14:     Returns:
15:         pd.DataFrame | None: DataFrame Pandas berisi data jika berhasil,
16:         None jika file tidak ditemukan atau error
17:         lain.
18:     """
19:     print(f"Mencoba membaca file CSV: {nama_file}")
20:     try:
21:         # Menggunakan pandas.read_csv untuk membaca file
22:         # Secara default, read_csv menganggap baris pertama sebagai
23:         # header
24:         # dan koma sebagai pemisah (separator).
25:         dataframe = pd.read_csv(nama_file)
26:         print("-> File CSV berhasil dibaca.")
27:         return dataframe
28:     except FileNotFoundError:
29:         # Menangani jika file tidak ditemukan
30:         print(f"-> ERROR: File '{nama_file}' tidak ditemukan!")
31:         return None
32:     except pd.errors.EmptyDataError:
33:         print(f"-> ERROR: File '{nama_file}' kosong.")
34:         return None
35:     except Exception as e:
36:         # Menangani error umum lainnya saat membaca CSV
37:         print(f"-> ERROR saat membaca file CSV: {type(e).__name__} - {e}")
38:         return None
39:
40: # --- Kode Utama ---
41: if __name__ == "__main__":
42:     print("--- Memulai Praktikum 2: Membaca CSV ---")
43:     # Panggil fungsi untuk membaca data
44:     df_lokasi = baca_data_lokasi(NAMA_FILE_CSV)
45:
46:     # Periksa apakah pembacaan berhasil (df_lokasi bukan None)
47:     if df_lokasi is not None:
48:         print("\n--- Inspeksi Awal DataFrame ---")
```

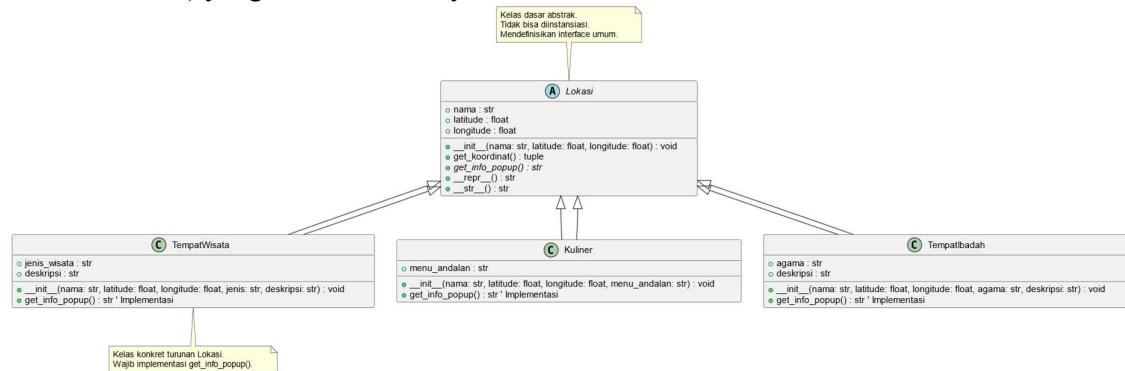
```
48:         # 1. Tampilkan 5 baris pertama data menggunakan head()
49:         print("\n1. Lima Baris Pertama (head()):")
50:         print(df_lokasi.head())
51:
52:         # 2. Tampilkan informasi ringkas tentang DataFrame menggunakan
info()
53:         #     (Jumlah baris, nama kolom, tipe data kolom, memori)
54:         print("\n2. Informasi DataFrame (info()):")
55:         df_lokasi.info()
56:
57:         # 3. Tampilkan dimensi DataFrame (jumlah baris, jumlah kolom)
58:         jumlah_baris, jumlah_kolom = df_lokasi.shape
59:         print(f"\n3. Dimensi Data:")
60:         print(f"    Jumlah Lokasi (Baris) : {jumlah_baris}")
61:         print(f"    Jumlah Atribut (Kolom): {jumlah_kolom}")
62:
63:         # 4. Tampilkan nama-nama kolom
64:         print(f"\n4. Nama Kolom:")
65:         print(list(df_lokasi.columns))
66:     else:
67:         print("\nTidak dapat melanjutkan inspeksi karena gagal membaca
file CSV.")
68:
69:     print("\n--- Praktikum 2 Selesai ---")
```

Observasi:

- Perhatikan bagaimana `import pandas as pd` digunakan untuk mengimpor library.
- Fungsi `pd.read_csv()` digunakan untuk membaca file. Jika berhasil, ia mengembalikan objek `DataFrame`.
- `try...except` digunakan untuk menangani kemungkinan `FileNotFoundError` atau error lain saat membaca file.
- Metode `.head()` pada `DataFrame` menampilkan beberapa baris awal.
- Metode `.info()` memberikan ringkasan struktur `DataFrame`, termasuk tipe data per kolom. Perhatikan tipe data untuk Latitude dan Longitude (kemungkinan `float64`) dan kolom lainnya (kemungkinan `object` yang berarti `string`).
- Atribut `.shape` memberikan jumlah baris dan kolom.
- Atribut `.columns` memberikan daftar nama kolom.

Praktikum 03: Mendesain dan Membuat Kelas OOP

Tujuan: Membuat struktur kelas menggunakan Pemrograman Berorientasi Objek (OOP) untuk merepresentasikan data lokasi geografis. Ini melibatkan pembuatan kelas dasar (abstrak) Lokasi dan beberapa kelas turunan konkret (misalnya TempatWisata, Kuliner, TempatIbadah) yang mewarisi darinya.



Penjelasan Kelas Diagram:

- **Kelas Lokasi:**
 - Ditandai sebagai kelas abstrak (abstract class).
 - Memiliki atribut dasar nama, latitude, longitude.
 - Memiliki metode konkret `__init__`, `get_koordinat`, `__repr__`, `__str__`.
 - Memiliki metode abstrak `get_info_popup()` yang harus diimplementasikan oleh subclass.
- **Kelas TempatWisata, Kuliner, TempatIbadah:**
 - Kelas konkret (class) yang mewarisi (extends atau `<|--`) dari Lokasi.
 - Masing-masing menambahkan atribut spesifiknya sendiri.
 - Masing-masing menyediakan implementasi konkret untuk metode abstrak `get_info_popup()`.
- **Catatan (Note):** Menjelaskan peran kelas abstrak Lokasi sebagai blueprint dan kewajiban kelas konkret seperti TempatWisata untuk mengimplementasikan metode abstrak.

Diagram ini menunjukkan struktur hierarki kelas yang dirancang untuk data SIG, dengan kelas abstrak sebagai dasar dan kelas konkret untuk tipe lokasi spesifik, serta penekanan pada metode abstrak sebagai kontrak interface.

Kode Program

```

01: class Buku:
02:     def __init__(self, judul, penulis, tahun, jml_halaman):
03:         self.judul = judul
04:         self.penulis = penulis
05:         self.tahun = tahun
06:         self.jml_halaman = max(0, jml_halaman) # Pastikan non-negatif
07:
08:     def __str__(self):
09:         # Versi singkat untuk kejelasan perbandingan
10:         return f'"{self.judul}" ({self.tahun})'
11:
12:     def __repr__(self):
13:         return f"Buku(judul='{self.judul}', penulis='{self.penulis}',
14:            tahun={self.tahun}, jml_halaman={self.jml_halaman})"
15:
16:     def __len__(self):
  
```

```

16:         return self.jml_halaman
17:
18:     # --- Implementasi Perbandingan ---
19:
20:     # __eq__: Dipanggil saat menggunakan operator ==
21:     def __eq__(self, other):
22:         """Membandingkan kesamaan dua objek Buku berdasarkan judul dan
penulis."""
23:         print(f"-> Memanggil __eq__: Membandingkan '{self.judul}' ==
'{'getattr(other, 'judul', '?')}'")
24:         # Cek apakah 'other' adalah instance dari kelas Buku
25:         if isinstance(other, Buku):
26:             # Logika kesamaan: judul DAN penulis harus sama
27:             return (self.judul == other.judul) and (self.penulis ==
other.penulis)
28:         # Jika tipe berbeda, kembalikan NotImplemented agar Python bisa
coba cara lain
29:         # atau menghasilkan TypeError jika tidak ada cara lain.
30:         return NotImplemented
31:
32:     # __lt__: Dipanggil saat menggunakan operator <
33:     def __lt__(self, other):
34:         """Membandingkan objek Buku berdasarkan tahun terbit (lebih kecil
dari)."""
35:         print(f"-> Memanggil __lt__: Membandingkan '{self.judul}'
({self.tahun}) < {'getattr(other, 'judul', '?')}' ({getattr(other, 'tahun',
'?')})")
36:         # Cek apakah 'other' adalah instance dari kelas Buku
37:         if isinstance(other, Buku):
38:             # Logika perbandingan: bandingkan tahun terbit
39:             return self.tahun < other.tahun
40:         # Jika tipe berbeda, operasi tidak didukung
41:         return NotImplemented
42:
43:     # Catatan: Jika Anda implementasi __eq__ dan __lt__, Python seringkali
44:     # bisa secara otomatis menyediakan implementasi untuk __ne__, __gt__,
45:     # __le__, __ge__ berdasarkan logika __eq__ dan __lt__.
46:     # Namun, untuk kontrol penuh atau optimasi, Anda bisa implementasikan
47:     # semuanya secara eksplisit jika perlu.
48:
49: # --- Kode Utama ---
50: if __name__ == "__main__":
51:     buku_A = Buku("Sejarah Jawa Kuno", "Prof. X", 1995, 450)
52:     buku_B = Buku("Teknologi AI", "Dr. Y", 2022, 300)
53:     buku_C = Buku("Sejarah Jawa Kuno", "Prof. X", 1995, 500) # Sama
judul&penulis dgn A
54:     buku_D = Buku("Pengantar Python", "Prof. X", 2018, 400)
55:
56:     print("\n--- Perbandingan Kesamaan (==) ---")
57:     print(f"'{buku_A.judul}' == '{buku_B.judul}' ? {buku_A == buku_B}")
# False (beda judul/penulis)
58:     print(f"'{buku_A.judul}' == '{buku_C.judul}' ? {buku_A == buku_C}")
# True (judul/penulis sama)
59:     print(f"'{buku_A.judul}' == 'Teks' ? {buku_A == 'Teks'}") #
False (beda tipe)
60:
61:     print("\n--- Perbandingan Kurang Dari (<) ---")
62:     print(f"{buku_A} < {buku_B} ? {buku_A < buku_B}") # True (1995 <
2022)
63:     print(f"{buku_B} < {buku_A} ? {buku_B < buku_A}") # False (2022 >
1995)

```

```
64:     print(f"{buku_A} < {buku_C} ? {buku_A < buku_C}") # False (1995 ==
1995)
65:     print(f"{buku_A} < {buku_D} ? {buku_A < buku_D}") # True (1995 <
2018)
66:
67:     print("\n--- Perbandingan Lain (Otomatis dari __lt__ dan __eq__) --
-")
68:     # Python menggunakan __lt__ dan __eq__ untuk menurunkan perbandingan
lain
69:     print(f"{buku_B} > {buku_A} ? {buku_B > buku_A}") # True (kebalikan
<)
70:     print(f"{buku_A} != {buku_B} ? {buku_A != buku_B}") # True (kebalikan
==)
71:
72:     print("\n--- Perbandingan dengan Tipe Lain ---")
73:     try:
74:         # Ini akan memanggil buku_A.__lt__(5), yang mengembalikan
NotImplemented
75:         # Python lalu mencoba 5.__gt__(buku_A), juga gagal -> TypeError
76:         hasil_error = buku_A < 5
77:         print(f"Hasil buku_A < 5 : {hasil_error}")
78:     except TypeError as e:
79:         print(f"Error saat membandingkan buku_A < 5: {e}")
```

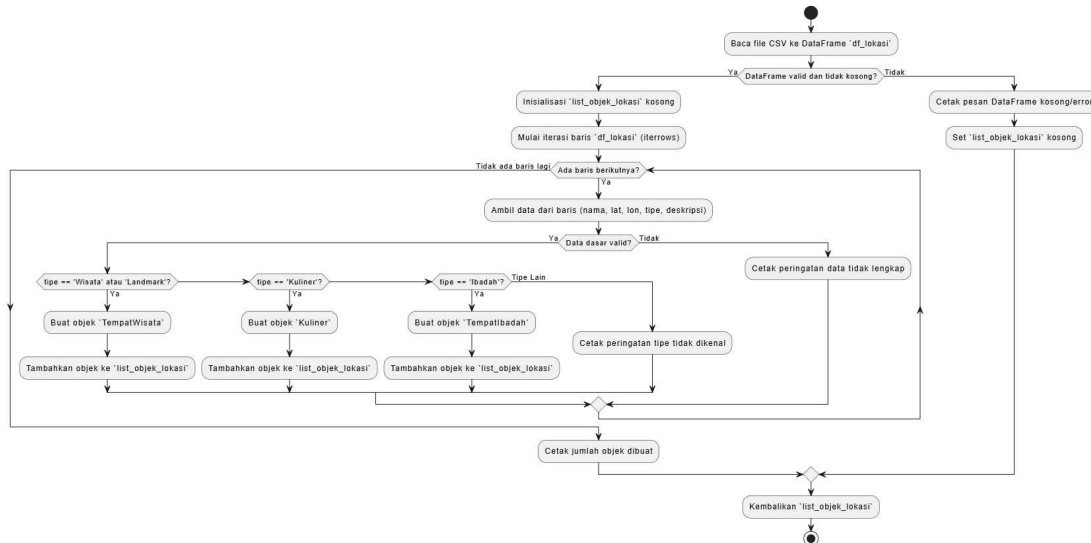
- **Observasi:**

- o Kelas `Lokasi` didefinisikan sebagai kelas abstrak (ABC) dengan metode `get_info_popup` sebagai metode abstrak (`@abstractmethod`). Ini berarti `Lokasi` tidak bisa diinstansiasi langsung dan semua kelas turunannya *harus* menyediakan implementasi untuk `get_info_popup`.
- o Kelas `TempatWisata`, `Kuliner`, dan `TempatIbadah` mewarisi dari `Lokasi`. Mereka memanggil konstruktor induk (`super().__init__`) dan menyediakan implementasi spesifik untuk `get_info_popup`. Format string di `get_info_popup` sengaja dibuat mirip HTML sederhana karena akan digunakan oleh Folium nanti.
- o Konstruktor `Lokasi` ditambahkan `try-except` sederhana untuk menangani jika input latitude/longitude tidak valid saat objek dibuat dari data CSV (antisipasi untuk Praktikum 4).

Praktikum 04: Membuat Objek dari Data Pandas

Tujuan: Mengiterasi baris-baris data dalam DataFrame Pandas yang telah dibaca dari file CSV, dan membuat instance objek dari kelas-kelas yang sesuai (TempatWisata, Kuliner, TempatIbadah, dll.) berdasarkan informasi tipe di setiap baris.

Prasyarat: Kode dari Praktikum 2 (fungsi `baca_data_lokasi`) dan Praktikum 3 (definisi kelas `Lokasi`, `TempatWisata`, `Kuliner`, `TempatIbadah`) diperlukan. File `lokasi_semarang.csv` harus ada.



Penjelasan Kelas Diagram (Diagram Aktivitas):

- **Diagram Aktivitas:** Menggambarkan alur kerja fungsi `buat_objek_lokasi_dari_df`.
- **Input:** Proses dimulai setelah DataFrame (`df_lokasi`) dibaca (hasil Praktikum 2).
- **Iterasi:** Menggunakan loop (`while`) untuk memproses setiap baris (`row`) dalam DataFrame.
- **Pengambilan Data:** Mengambil nilai dari kolom-kolom yang relevan pada setiap baris.
- **Validasi Dasar:** Ada pemeriksaan awal untuk kelengkapan data penting.
- **Percabangan Tipe:** Struktur `if/elif/else` digunakan untuk menentukan kelas mana yang akan diinstansiasi berdasarkan nilai kolom 'Tipe'.
- **Pembuatan Objek:** Instance dari kelas yang sesuai (`TempatWisata`, `Kuliner`, `TempatIbadah`) dibuat.
- **Penambahan ke List:** Objek yang baru dibuat ditambahkan ke `list_objek_lokasi`.
- **Output:** Fungsi mengembalikan list yang berisi semua objek yang berhasil dibuat.

Diagram ini menyoroti logika inti dari praktikum ini: transformasi data tabular dari DataFrame menjadi representasi objek yang terstruktur menggunakan kelas-kelas yang telah didefinisikan sebelumnya.

Kode Program:

```

001: import pandas as pd
002: from abc import ABC, abstractmethod # Diperlukan untuk definisi kelas
003:
004: # --- Definisi Kelas (Salin dari Praktikum 3) ---
005: class Lokasi(ABC):
006:     def __init__(self, nama: str, latitude: float, longitude: float):
007:         self.nama = str(nama) if nama else "Tanpa Nama"
008:         try:
009:             self.latitude = float(latitude)
010:             self.longitude = float(longitude)
011:         except (ValueError, TypeError, SystemError):
  
```

```

012:         # print(f" -> Peringatan: Koordinat tidak valid untuk
' {self.nama}'. Set ke (0.0, 0.0).")
013:         self.latitude = 0.0
014:         self.longitude = 0.0
015:         def get_koordinat(self) -> tuple: return (self.latitude,
self.longitude)
016:         @abstractmethod
017:         def get_info_popup(self) -> str: pass
018:         def __repr__(self) -> str: return
f"{type(self).__name__} (nama='{self.nama}', lat={self.latitude:.4f},
lon={self.longitude:.4f})"
019:         def __str__(self) -> str: return f"{self.nama}
[{type(self).__name__}]"
020:
021: class TempatWisata(Lokasi):
022:     def __init__(self, nama: str, latitude: float, longitude: float,
jenis: str, deskripsi: str):
023:         super().__init__(nama, latitude, longitude); self.jenis_wisata
= str(jenis) if jenis else "Umum"; self.deskripsi = str(deskripsi) if
deskripsi else "Tidak ada deskripsi."
024:         def get_info_popup(self) -> str: return
f"<h4><b>{self.nama}</b></h4><i>{self.jenis_wisata}</i><br><br>{self.deskri
psi}<br><br>Koordinat: ({self.latitude:.4f}, {self.longitude:.4f})"
025:
026: class Kuliner(Lokasi):
027:     def __init__(self, nama: str, latitude: float, longitude: float,
menu_andalan: str):
028:         super().__init__(nama, latitude, longitude); self.menu_andalan
= str(menu_andalan) if menu_andalan else "Tidak diketahui"
029:         def get_info_popup(self) -> str: return
f"<h4><b>{self.nama}</b></h4><i>Kuliner</i><br><br>Menu Andalan:
{self.menu_andalan}<br><br>Koordinat: ({self.latitude:.4f},
{self.longitude:.4f})"
030:
031: class TempatIbadah(Lokasi):
032:     def __init__(self, nama: str, latitude: float, longitude: float,
agama: str = "Umum", deskripsi: str = ""):
033:         super().__init__(nama, latitude, longitude); self.agama =
str(agama) if agama else "Umum"; self.deskripsi = str(deskripsi) if deskripsi
else "Tempat Ibadah"
034:         def get_info_popup(self) -> str: return
f"<h4><b>{self.nama}</b></h4><i>Tempat Ibadah
({self.agama})</i><br><br>{self.deskripsi}<br><br>Koordinat:
({self.latitude:.4f}, {self.longitude:.4f})"
035:
036: # --- Fungsi baca data (Salin dari Praktikum 2) ---
037: def baca_data_lokasi(nama_file: str) -> pd.DataFrame | None:
038:     # print(f"Mencoba membaca file CSV: {nama_file}") # Kurangi verbosity
039:     try: dataframe = pd.read_csv(nama_file); return dataframe
040:     except FileNotFoundError: print(f"ERROR: File '{nama_file}' tidak
ditemukan!"); return None
041:     except Exception as e: print(f"ERROR saat membaca file CSV:
{type(e).__name__} - {e}"); return None
042:
043: # --- Fungsi Inti Praktikum Ini ---
044: def buat_objek_lokasi_dari_df(dataframe: pd.DataFrame) -> list:
045:     """
046:     Mengiterasi DataFrame Pandas dan membuat list berisi objek-objek
047:     Lokasi (atau turunannya) berdasarkan data di setiap baris.
048:
049:     Args:

```

```

050:         dataframe (pd.DataFrame): DataFrame yang berisi data lokasi
051:                                     dengan kolom 'Nama', 'Latitude',
'Longitude',
052:                                     'Tipe', 'Deskripsi'.
053:
054:     Returns:
055:         list: List berisi instance objek Lokasi atau turunannya.
056:         ""
057:         list_objek_lokasi = []
058:         if dataframe is None or dataframe.empty:
059:             print("DataFrame kosong atau None, tidak ada objek dibuat.")
060:             return list_objek_lokasi
061:
062:         print("\nMembuat objek dari DataFrame...")
063:         # Iterasi setiap baris dalam DataFrame
064:         for index, row in dataframe.iterrows():
065:             # Ambil data dari setiap kolom di baris saat ini
066:             # Gunakan .get() dengan default value untukantisipasi kolom
hilang
067:             nama = row.get('Nama', None)
068:             lat = row.get('Latitude', None)
069:             lon = row.get('Longitude', None)
070:             tipe = row.get('Tipe', 'Lainnya') # Default jika kolom Tipe
tidak ada
071:             deskripsi = row.get('Deskripsi', '') # Default string kosong
072:
073:             objek = None
074:             # Lakukan pengecekan tipe data dasar sebelum membuat objek
075:             if nama is None or lat is None or lon is None:
076:                 print(f"    -> Melewati baris {index}: Data
Nama/Latitude/Longitude tidak lengkap.")
077:                 continue # Lanjut ke baris berikutnya
078:
079:             # Membuat objek berdasarkan nilai di kolom 'Tipe'
080:             try:
081:                 if 'Wisata' in tipe or tipe == 'Landmark':
082:                     objek = TempatWisata(nama, lat, lon, tipe, deskripsi)
083:                 elif tipe == 'Kuliner':
084:                     # Menggunakan kolom 'Deskripsi' sebagai 'menu_andalan'
untuk contoh ini
085:                     objek = Kuliner(nama, lat, lon, deskripsi)
086:                 elif 'Ibadah' in tipe:
087:                     # Bisa ekstrak info agama dari tipe jika ada, atau set
default
088:                     agama_info = "Umum" # Default
089:                     if "Islam" in tipe: agama_info="Islam"
090:                     elif "Kristen" in tipe: agama_info="Kristen"
091:                     elif "Klenteng" in tipe: agama_info="Tridharma"
092:                     # Dst...
093:                     objek = TempatIbadah(nama, lat, lon, agama_info,
deskripsi)
094:                 else:
095:                     print(f"    -> Peringatan: Tipe '{tipe}' untuk '{nama}'
tidak dikenali. Tidak membuat objek spesifik.")
096:                     # Karena Lokasi abstrak, kita tidak bisa membuat objek
Lokasi generik.
097:                     # Jika Lokasi tidak abstrak, bisa: objek = Lokasi(nama,
lat, lon)
098:
099:                 if objek:
100:                     list_objek_lokasi.append(objek)

```



```

101:                                # print(f"  -> Objek {type(objek).__name__} untuk
'{nama}' dibuat.")
102:
103:         except Exception as e:
104:             # Tangani error saat pembuatan objek (misal konversi tipe
gagal di __init__)
105:             print(f"  -> GAGAL membuat objek untuk '{nama}' di baris
{index}: {e}")
106:
107:
108:     print(f"Total {len(list_objek_lokasi)} objek lokasi berhasil dibuat
dari {len(dataframe)} baris data.")
109:     return list_objek_lokasi
110:
111: # --- Kode Utama ---
112: if __name__ == "__main__":
113:     NAMA_FILE_CSV = "lokasi_semarang.csv"
114:
115:     print("--- Memulai Praktikum 4: Membuat Objek dari Data Pandas ---
")
116:     # 1. Baca data CSV
117:     df_lokasi = baca_data_lokasi(NAMA_FILE_CSV)
118:
119:     # 2. Buat list objek dari DataFrame
120:     list_semua_lokasi = buat_objek_lokasi_dari_df(df_lokasi)
121:
122:     # 3. Tampilkan hasil (representasi objek)
123:     print("\n--- Daftar Objek Lokasi yang Berhasil Dibuat ---")
124:     if list_semua_lokasi:
125:         for idx, lok in enumerate(list_semua_lokasi):
126:             # repr() akan menunjukkan tipe objeknya (TempatWisata,
Kuliner, dll.)
127:             print(f"{idx+1}. {repr(lok)}")
128:     else:
129:         print("Tidak ada objek lokasi yang dibuat.")
130:
131:     print("\n--- Praktikum 4 Selesai ---")

```

Observasi:

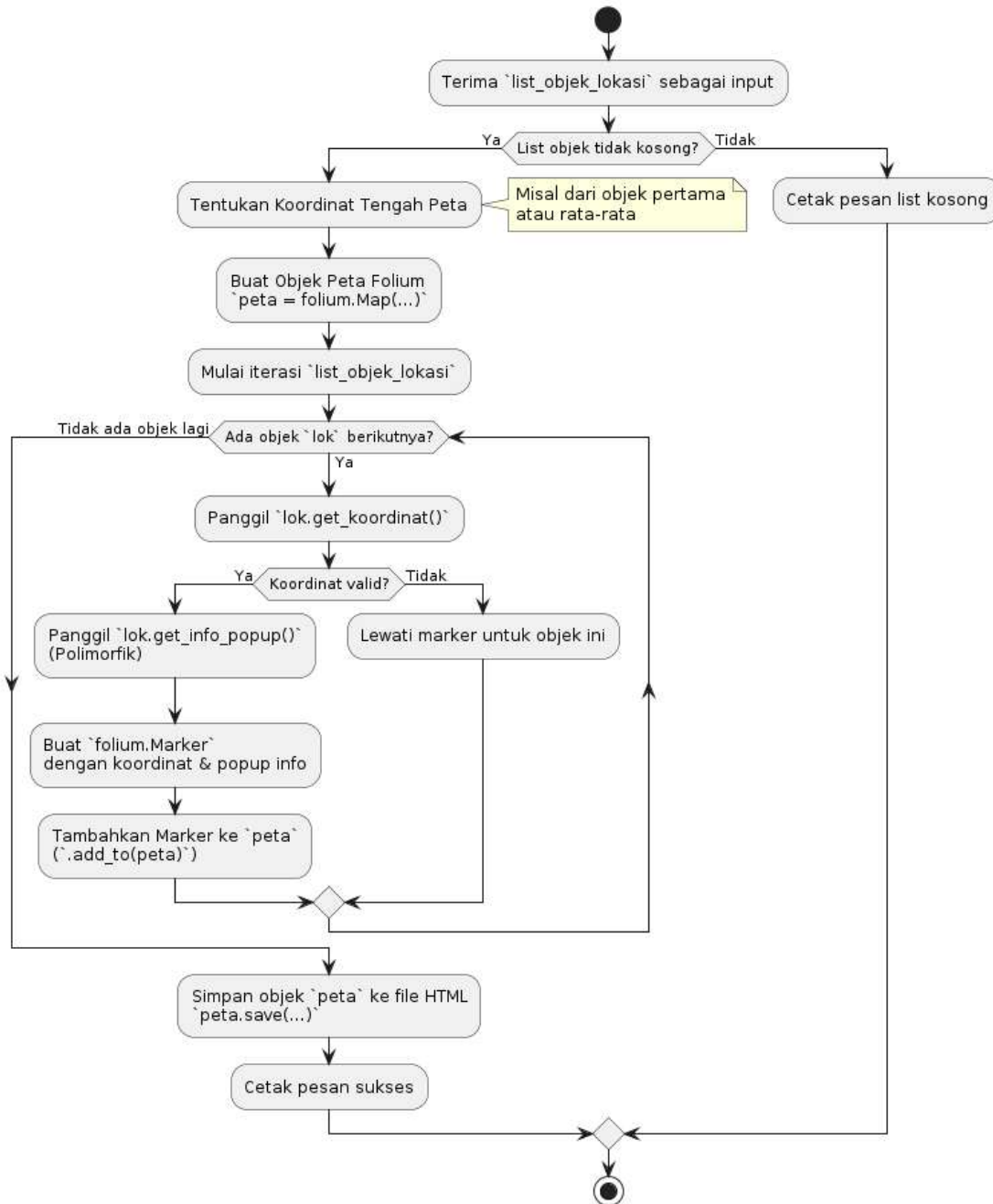
- Fungsi `buat_objek_lokasi_dari_df` menerima `DataFrame` sebagai input.
- Ia menggunakan `dataframe.iterrows()` untuk mengulang setiap baris. `row` adalah objek `Series` Pandas yang memungkinkan akses data per kolom (`row['Nama']`, `row['Tipe']`, dll.).
- Struktur `if/elif/else` digunakan untuk memeriksa nilai kolom 'Tipe'. Berdasarkan nilai ini, objek dari kelas yang sesuai (`TempatWisata`, `Kuliner`, `TempatIbadah`) diinstansiasi.
- Objek yang berhasil dibuat ditambahkan ke `list_objek_lokasi`.
- Penanganan error ditambahkan untuk kasus data tidak lengkap atau tipe tidak dikenali.
- Output akhir menunjukkan representasi (`repr`) dari setiap objek dalam list, memperlihatkan bahwa objek dari kelas yang berbeda telah berhasil dibuat berdasarkan data CSV.

Praktikum 05: Visualisasi Peta dengan Folium

Tujuan: Menggunakan library Folium untuk membuat peta HTML interaktif dan menambahkan marker (penanda) untuk setiap objek lokasi yang telah dibuat pada praktikum sebelumnya, memanfaatkan informasi (koordinat dan popup) yang diambil secara polimorfik dari setiap objek.

Prasyarat: Kode dari Praktikum 4 (termasuk definisi kelas dan fungsi `baca_data_lokasi`, `buat_objek_lokasi_dari_df`) diperlukan. Library `folium` harus sudah terinstal (`pip install folium`). File `lokasi_semarang.csv` harus ada.

Proses Membuat Peta Lokasi Folium



Penjelasan Kelas Diagram (Diagram Aktivitas):

- **Diagram Aktivitas:** Menggambarkan alur langkah dalam fungsi `buat_peta_lokasi_folium`.
- **Input:** Menerima list objek lokasi (hasil dari Praktikum 4).
- **Inisialisasi Peta:** Menentukan titik tengah dan membuat objek `folium.Map`.
- **Iterasi Objek:** Melakukan loop untuk setiap objek `lok` dalam list input.
- **Pengambilan Data Objek:** Memanggil metode `get_koordinat()` dan `get_info_popup()` pada setiap objek `lok`. Pemanggilan `get_info_popup()` bersifat polimorfik.
- **Pembuatan Marker:** Membuat `folium.Marker` menggunakan data yang diperoleh dari objek.
- **Penambahan ke Peta:** Menambahkan marker yang sudah dibuat ke objek peta.
- **Penyimpanan:** Setelah loop selesai, menyimpan objek peta ke file HTML.
- **Penanganan Kondisi:** Termasuk pemeriksaan jika list input kosong dan jika koordinat objek tidak valid.

Diagram ini menunjukkan bagaimana data yang sudah terstruktur dalam bentuk objek OOP digunakan untuk menghasilkan visualisasi geografis interaktif menggunakan library Folium, dengan memanfaatkan polimorfisme untuk mendapatkan detail spesifik dari setiap objek.

Kode Program:

```
001: # !pip install folium pandas # jalankan kode ini jika Folium dan Pandas
    belum ada
002: import pandas as pd
003: import folium # Impor library Folium
004: from abc import ABC, abstractmethod # Impor ABC dan abstractmethod
005:
006: # --- Definisi Kelas (Salin dari Praktikum 3/4) ---
007: class Lokasi(ABC):
008:     def __init__(self, nama: str, latitude: float, longitude: float):
009:         self.nama = str(nama) if nama else "Tanpa Nama"
010:         try:
011:             self.latitude, self.longitude = float(latitude),
float(longitude)
012:         except ValueError:
013:             self.latitude, self.longitude = 0.0, 0.0
014:         def get_koordinat(self) -> tuple: return (self.latitude,
self.longitude)
015:         @abstractmethod
016:         def get_info_popup(self) -> str: pass
017:         def __repr__(self) -> str: return
f"{type(self).__name__} (nama='{self.nama}', lat={self.latitude:.4f},
lon={self.longitude:.4f})"
018:         def __str__(self) -> str: return f"{self.nama}
[{type(self).__name__}]"
019:
020: class TempatWisata(Lokasi):
021:     def __init__(self, nama: str, latitude: float, longitude: float,
jenis: str, deskripsi: str): super().__init__(nama, latitude, longitude);
self.jenis_wisata=str(jenis) if jenis else "Umum";
self.deskripsi=str(deskripsi) if deskripsi else "Tidak ada deskripsi."
022:     def get_info_popup(self) -> str: return
f"<h4><b>{self.nama}</b></h4><i>{self.jenis_wisata}</i><br><br>{self.deskri
psi}<br><br>Koordinat: ({self.latitude:.4f}, {self.longitude:.4f})"
023:
024: class Kuliner(Lokasi):
025:     def __init__(self, nama: str, latitude: float, longitude: float,
menu_andalan: str): super().__init__(nama, latitude, longitude);
self.menu_andalan=str(menu_andalan) if menu_andalan else "Tidak diketahui"
```

```

026:         def get_info_popup(self) -> str: return
f"<h4><b>{self.nama}</b></h4><i>Kuliner</i><br><br>Menu
Andalan:
{self.menu_andalan}<br><br>Koordinat:
({self.latitude:.4f},
{self.longitude:.4f})"
027:
028: class TempatIbadah(Lokasi):
029:     def __init__(self, nama: str, latitude: float, longitude: float,
agama: str = "Umum", deskripsi: str = ""): super().__init__(nama, latitude,
longitude); self.agama=agama if agama else "Umum";
self.deskripsi=deskripsi if deskripsi else "Tempat Ibadah"
030:     def get_info_popup(self) -> str: return
f"<h4><b>{self.nama}</b></h4><i>Tempat
Ibadah
({self.agama})</i><br><br>{self.deskripsi}<br><br>Koordinat:
({self.latitude:.4f}, {self.longitude:.4f})"
031:
032: # --- Fungsi baca data dan buat objek (Salin dari Praktikum 4) ---
033: def baca_data_lokasi(nama_file: str) -> pd.DataFrame | None:
034:     try: dataframe = pd.read_csv(nama_file); return dataframe
035:     except FileNotFoundError: print(f"ERROR: File '{nama_file}' tidak
ditemukan!"); return None
036:     except Exception as e: print(f"ERROR saat membaca file CSV:
{type(e).__name__} - {e}"); return None
037:
038: def buat_objek_lokasi_dari_df(dataframe: pd.DataFrame) -> list:
039:     list_objek_lokasi = [];
040:     if dataframe is None or dataframe.empty: return list_objek_lokasi
041:     # print("\nMembuat objek dari DataFrame...") # Kurangi verbosity
042:     for index, row in dataframe.iterrows():
043:         nama=row.get('Nama',None); lat=row.get('Latitude',None);
lon=row.get('Longitude',None); tipe=row.get('Tipe','Lainnya');
deskripsi=row.get('Deskripsi','')
044:         objek = None
045:         if nama is None or lat is None or lon is None: continue
046:         try:
047:             if 'Wisata' in tipe or tipe == 'Landmark':
objek=TempatWisata(nama, lat, lon, tipe, deskripsi)
048:             elif tipe == 'Kuliner': objek=Kuliner(nama, lat, lon,
deskripsi)
049:             elif 'Ibadah' in tipe: agama_info="Umum";
objek=TempatIbadah(nama, lat, lon, agama_info, deskripsi)
050:             if objek: list_objek_lokasi.append(objek)
051:             except Exception as e: print(f" -> GAGAL membuat objek untuk
'{nama}' di baris {index}: {e}")
052:             # print(f"Total {len(list_objek_lokasi)} objek lokasi berhasil
dibuat...") # Kurangi verbosity
053:             return list_objek_lokasi
054:
055: # --- Fungsi Inti Praktikum Ini ---
056: def buat_peta_lokasi_folium(list_objek: list, file_output: str =
"peta_lokasi.html"):
057:     """
058:     Membuat peta Folium interaktif dengan marker untuk setiap objek
059:     dalam list_objek.
060:
061:     Args:
062:         list_objek (list): List berisi instance objek turunan Lokasi.
063:         file_output (str): Nama file HTML untuk menyimpan peta.
064:     """
065:     if not list_objek:
066:         print("Tidak ada objek lokasi untuk dipetakan.")
067:         return

```

```
068:
069:     print(f"\nMemulai pembuatan peta Folium dari {len(list_objek)}
lokasi...")
070:
071:     # 1. Tentukan titik tengah peta (misal: lokasi pertama atau rata-
rata)
072:     try:
073:         lat_tengah = list_objek[0].latitude
074:         lon_tengah = list_objek[0].longitude
075:     except IndexError:
076:         lat_tengah, lon_tengah = -6.9929, 110.4200 # Default Semarang
jika list kosong
077:
078:     # 2. Buat objek peta Folium
079:     #     zoom_start menentukan level zoom awal (angka lebih besar =
lebih dekat)
080:     peta = folium.Map(location=[lat_tengah, lon_tengah], zoom_start=13,
tiles="OpenStreetMap")
081:     print(f"    -> Objek peta dibuat, berpusat di ({lat_tengah:.4f},
{lon_tengah:.4f})")
082:
083:     # 3. Tambahkan marker untuk setiap lokasi dalam list
084:     jumlah_marker_valid = 0
085:     for lok in list_objek:
086:         koordinat = lok.get_koordinat()
087:
088:         # Pastikan koordinat valid (bukan 0.0, 0.0 dari error sebelumnya)
089:         if koordinat != (0.0, 0.0):
090:             # Ambil info popup secara polimorfik dari objek
091:             # Metode get_info_popup() akan memanggil implementasi yang
sesuai
092:             # (TempatWisata, Kuliner, TempatIbadah)
093:             info_popup_html = lok.get_info_popup()
094:
095:             # Buat objek Marker dan tambahkan ke peta
096:             folium.Marker(
097:                 location=koordinat,                                # Koordinat
marker
098:                 popup=folium.Popup(info_popup_html, max_width=300), #
Konten popup saat diklik
099:                 tooltip=lok.nama                                # Teks saat
hover
100:                 # icon=folium.Icon(color='blue', icon='info-sign') #
Contoh kustomisasi ikon
101:             ).add_to(peta)
102:             jumlah_marker_valid += 1
103:         else:
104:             print(f"    -> Melewati marker untuk '{lok.nama}' karena
koordinat tidak valid.")
105:
106:     # 4. Simpan peta ke file HTML
107:     try:
108:         peta.save(file_output)
109:         print(f"\n-> Peta berhasil dibuat dan disimpan sebagai
'{file_output}'.")
110:         print(f"    Total marker ditambahkan: {jumlah_marker_valid}")
111:     except Exception as e:
112:         print(f"\nERROR saat menyimpan peta Folium: {type(e).__name__}
- {e}")
113:
114: # --- Kode Utama ---
```

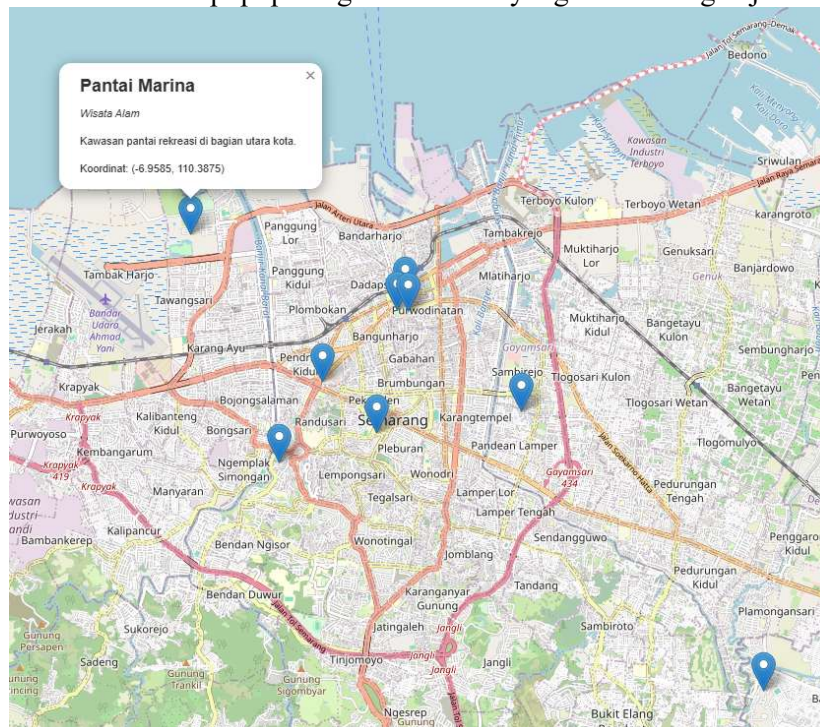
```

115: if __name__ == "__main__":
116:     NAMA_FILE_CSV = "lokasi_semarang.csv"
117:     NAMA_FILE_PETA = "peta_interaktif_semarang.html"
118:
119:     print("--- Memulai Praktikum 5: Visualisasi Peta dengan Folium ---")
120:
121:     # 1. Baca data CSV
122:     df_lokasi = baca_data_lokasi(NAMA_FILE_CSV)
123:
124:     # 2. Buat list objek dari DataFrame
125:     list_semua_lokasi = buat_objek_lokasi_dari_df(df_lokasi)
126:
127:     # 3. Buat peta dari list objek
128:     buat_peta_lokasi_folium(list_semua_lokasi, NAMA_FILE_PETA)
129:
130:     print(f"\nSilakan buka file '{NAMA_FILE_PETA}' di browser Anda untuk melihat hasilnya.")
131:
132:     print("\n--- Praktikum 5 Selesai ---")

```

Observasi:

- Pastikan library `folium` sudah terinstal.
- Kode ini pertama-tama membuat objek peta dasar `folium.Map` dengan lokasi tengah dan zoom awal.
- Kemudian, ia melakukan iterasi pada `list_objek` yang berisi instance `TempatWisata`, `Kuliner`, dll.
- Untuk setiap objek `lok`, metode `lok.get_koordinat()` dan `lok.get_info_popup()` dipanggil. Karena `get_info_popup` di-override di setiap subclass, pemanggilan ini bersifat polimorfik dan menghasilkan konten popup yang sesuai dengan tipe lokasi.
- `folium.Marker` dibuat dengan koordinat dan informasi popup tersebut, lalu ditambahkan ke peta menggunakan `.add_to(peta)`.
- Terakhir, `peta.save()` menyimpan keseluruhan peta interaktif sebagai file HTML. Buka file HTML ini di browser web Anda. Anda seharusnya bisa mengklik setiap marker untuk melihat popup dengan informasi yang sesuai dengan jenis lokasinya.

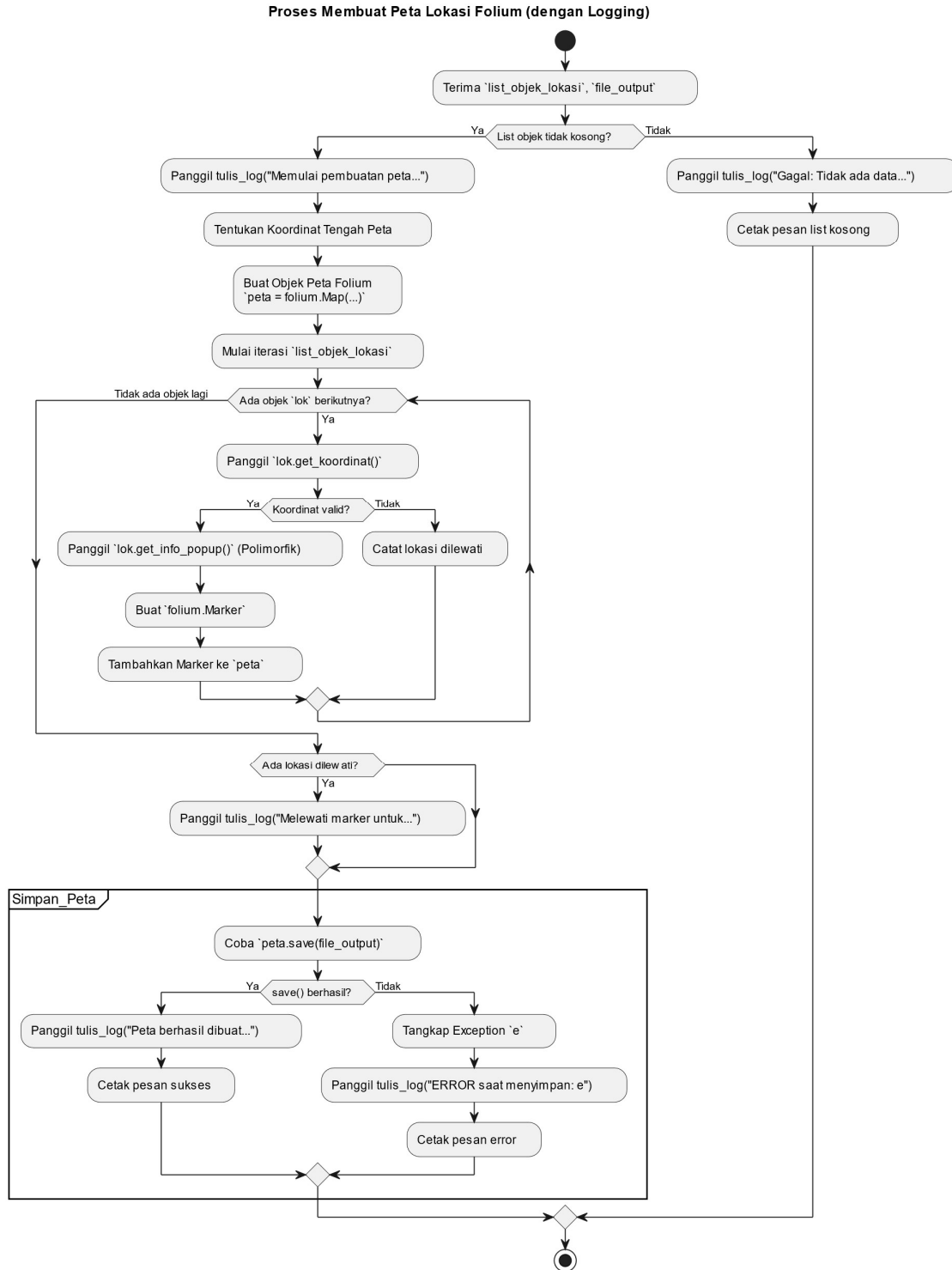


Praktikum 06: File Handling Tambahan (Menyimpan Log)

Tujuan: Menggunakan file handling dasar (mode 'a' - append) untuk mencatat (log) informasi sederhana ke dalam file teks setiap kali proses pembuatan peta dijalankan, baik berhasil maupun gagal.

Prasyarat: Kode dari Praktikum 5 (termasuk definisi kelas, fungsi baca data, fungsi buat objek, dan fungsi buat peta) diperlukan. Library `pandas` dan `folium` harus terinstal. File `lokasi_semarang.csv` harus ada.

Tujuan: Menambahkan *type hints* pada parameter fungsi/metode dan nilai kembali sebagai bentuk dokumentasi tambahan yang eksplisit dan untuk mendukung alat analisis statis.



Penjelasan Kelas Diagram (Diagram Aktivitas):

- **Diagram Aktivitas:** Menggambarkan alur fungsi `buat_peta_lokasi_folium` yang dimodifikasi.
- **Pemanggilan `tulis_log`:** Aktivitas baru ditambahkan untuk merepresentasikan pemanggilan fungsi `tulis_log` pada titik-titik kritis:
 - Saat proses pembuatan peta dimulai.

- Jika ada lokasi yang dilewati karena koordinat tidak valid.
- Setelah berhasil menyimpan peta (`peta.save()`).
- Jika terjadi error saat menyimpan peta.
- **Partisi Simpan_Peta:** Mengelompokkan langkah-langkah yang terkait dengan penyimpanan peta, termasuk penanganan `try-except` saat menyimpan.
- **Alur Lain:** Alur utama untuk membuat peta dan marker tetap sama seperti pada Praktikum 5.

Diagram ini menunjukkan bagaimana fungsionalitas logging (menggunakan file handling dengan mode append) diintegrasikan ke dalam alur kerja utama pembuatan peta untuk mencatat keberhasilan atau kegagalan proses.

Kode Program:

```

001: import pandas as pd
002: import folium
003: import datetime # Untuk timestamp log
004: import time      # Hanya untuk simulasi di kelas jika diperlukan
005: from abc import ABC, abstractmethod # Impor ABC dan abstractmethod
006:
007: # --- Definisi Kelas (Salin dari Praktikum 3/4) ---
008: # (Definisi Lokasi, TempatWisata, Kuliner, TempatIbadah ada di sini)
009: class Lokasi(ABC):
010:     def __init__(self, nama: str, latitude: float, longitude: float):
011:         self.nama = str(nama) if nama else "Tanpa Nama"
012:         try:
013:             self.latitude = float(latitude)
014:             self.longitude = float(longitude)
015:         except ValueError:
016:             self.latitude = 0.0
017:             self.longitude = 0.0
018:         def get_koordinat(self) -> tuple: return (self.latitude,
self.longitude)
019:         @abstractmethod
020:         def get_info_popup(self) -> str: pass
021:         def __repr__(self) -> str: return
f"{type(self).__name__} (nama='{self.nama}', lat={self.latitude:.4f},
lon={self.longitude:.4f})"
022:         def __str__(self) -> str: return f"{self.nama}
[{type(self).__name__}]"
023:
024: class TempatWisata(Lokasi):
025:     def __init__(self, nama: str, latitude: float, longitude: float,
jenis: str, deskripsi: str): super().__init__(nama, latitude, longitude);
self.jenis_wisata=str(jenis) if jenis else "Umum";
self.deskripsi=str(deskripsi) if deskripsi else "Tidak ada deskripsi."
026:     def get_info_popup(self) -> str: return
f"<h4><b>{self.nama}</b></h4><i>{self.jenis_wisata}</i><br><br>{self.deskri
psi}<br><br>Koordinat: ({self.latitude:.4f}, {self.longitude:.4f})"
027:
028: class Kuliner(Lokasi):
029:     def __init__(self, nama: str, latitude: float, longitude: float,
menu_andalan: str): super().__init__(nama, latitude, longitude);
self.menu_andalan=str(menu_andalan) if menu_andalan else "Tidak diketahui"
030:     def get_info_popup(self) -> str: return
f"<h4><b>{self.nama}</b></h4><i>Kuliner</i><br><br>Menu Andalan:
{self.menu_andalan}<br><br>Koordinat: ({self.latitude:.4f},
{self.longitude:.4f})"
031:
032: class TempatIbadah(Lokasi):

```

```

033:     def __init__(self, nama: str, latitude: float, longitude: float,
agama: str = "Umum", deskripsi: str = ""): super().__init__(nama, latitude,
longitude); self.agama=str(agama) if agama else "Umum";
self.deskripsi=str(deskripsi) if deskripsi else "Tempat Ibadah"
034:         def get_info_popup(self) -> str: return
f"<h4><b>{self.nama}</b></h4><i>Tempat Ibadah
({self.agama})</i><br><br>{self.deskripsi}<br><br>Koordinat:
({self.latitude:.4f}, {self.longitude:.4f})"
035:
036:
037: # --- Fungsi baca data dan buat objek (Salin dari Praktikum 4) ---
038: def baca_data_lokasi(nama_file: str) -> pd.DataFrame | None:
039:     try: dataframe = pd.read_csv(nama_file); return dataframe
040:     except FileNotFoundError: print(f"ERROR: File '{nama_file}' tidak
ditemukan!"); return None
041:     except Exception as e: print(f"ERROR saat membaca file CSV:
{type(e).__name__} - {e}"); return None
042:
043: def buat_objek_lokasi_dari_df(dataframe: pd.DataFrame) -> list:
044:     list_objek_lokasi = [];
045:     if dataframe is None or dataframe.empty: return list_objek_lokasi
046:     for index, row in dataframe.iterrows():
047:         nama=row.get('Nama',None); lat=row.get('Latitude',None);
lon=row.get('Longitude',None); tipe=row.get('Tipe','Lainnya');
deskripsi=row.get('Deskripsi','')
048:         objek = None
049:         if nama is None or lat is None or lon is None: continue
050:         try:
051:             if 'Wisata' in tipe or tipe == 'Landmark':
objek=TempatWisata(nama, lat, lon, tipe, deskripsi)
052:             elif tipe == 'Kuliner': objek=Kuliner(nama, lat, lon,
deskripsi)
053:             elif 'Ibadah' in tipe: agama_info="Umum";
objek=TempatIbadah(nama, lat, lon, agama_info, deskripsi)
054:             if objek: list_objek_lokasi.append(objek)
055:             except Exception as e: print(f" -> GAGAL membuat objek untuk
'{nama}' di baris {index}: {e}")
056:         return list_objek_lokasi
057:
058: # --- Fungsi untuk Menulis Log ---
059: def tulis_log(pesan: str, file_log: str = "proses_peta.log"):
060:     """Menulis pesan log ke file dengan timestamp, menggunakan mode
append."""
061:     timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
062:     try:
063:         # Menggunakan 'with open' memastikan file otomatis ditutup
064:         # Mode 'a' (append) untuk menambahkan di akhir file
065:         # encoding='utf-8' disarankan untuk kompatibilitas
066:         with open(file_log, 'a', encoding='utf-8') as f:
067:             f.write(f"[{timestamp}] {pesan}\n")
068:             # print(f" -> Log ditulis ke {file_log}") # Optional: konfirmasi
penulisan log
069:     except IOError as e:
070:         # Tangani jika ada error saat menulis ke file log
071:         print(f"ERROR: Gagal menulis ke file log '{file_log}': {e}")
072:
073: # --- Fungsi buat peta (Dimodifikasi untuk Logging) ---
074: def buat_peta_lokasi_folium(list_objek: list, file_output: str =
"peta_lokasi.html"):
075:     """
076:     Membuat peta Folium, menambahkan marker, menyimpan ke HTML,

```

```
077:     dan menulis log proses.
078:     """
079:     nama_fungsi = "buat_peta_lokasi_folium" # Untuk log
080:
081:     if not list_objek:
082:         pesan_log = f"[{nama_fungsi}] Gagal: Tidak ada data lokasi untuk
dipetakan."
083:         print(pesan_log)
084:         tulis_log(pesan_log) # Log kegagalan
085:         return
086:
087:         print(f"\n[{nama_fungsi}] Memulai pembuatan peta dari
{len(list_objek)} lokasi...")
088:         tulis_log(f"[{nama_fungsi}] Memulai pembuatan peta '{file_output}'
dengan {len(list_objek)} lokasi.")
089:
090:         try:
091:             lat_tengah = list_objek[0].latitude; lon_tengah =
list_objek[0].longitude
092:         except IndexError:
093:             lat_tengah, lon_tengah = -6.9929, 110.4200 # Default Semarang
094:             peta = folium.Map(location=[lat_tengah, lon_tengah], zoom_start=12)
095:
096:             jumlah_marker = 0
097:             lokasi_dilewati = []
098:             for lok in list_objek:
099:                 koordinat = lok.get_koordinat()
100:                 if koordinat != (0.0, 0.0):
101:                     info_popup_html = lok.get_info_popup()
102:                     folium.Marker(
103:                         location=koordinat, popup=folium.Popup(info_popup_html,
max_width=300), tooltip=lok.nama
104:                     ).add_to(peta)
105:                     jumlah_marker += 1
106:                 else:
107:                     lokasi_dilewati.append(lok.nama)
108:
109:             if lokasi_dilewati:
110:                 pesan_lewat = f"[{nama_fungsi}] Melewati marker untuk: {'',
'.join(lokasi_dilewati)} (koordinat tidak valid).".
111:                 print(f" -> Peringatan: {pesan_lewat}")
112:                 tulis_log(pesan_lewat)
113:
114:             # Simpan peta dan tulis log
115:             try:
116:                 peta.save(file_output)
117:                 pesan_sukses = f"[{nama_fungsi}] Peta '{file_output}' berhasil
dibuat dengan {jumlah_marker} marker."
118:                 print(f"-> {pesan_sukses}")
119:                 tulis_log(pesan_sukses) # Log keberhasilan
120:             except Exception as e:
121:                 pesan_error = f"[{nama_fungsi}] ERROR saat menyimpan peta
'{file_output}': {type(e).__name__} - {e}"
122:                 print(f"-> {pesan_error}")
123:                 tulis_log(pesan_error) # Log kegagalan
124:
125: # --- Kode Utama ---
126: if __name__ == "__main__":
127:     NAMA_FILE_CSV = "lokasi_semarang.csv"
128:     NAMA_FILE_PETA = "peta_interaktif_semarang.html"
129:     FILE_LOG = "proses_peta.log"
```

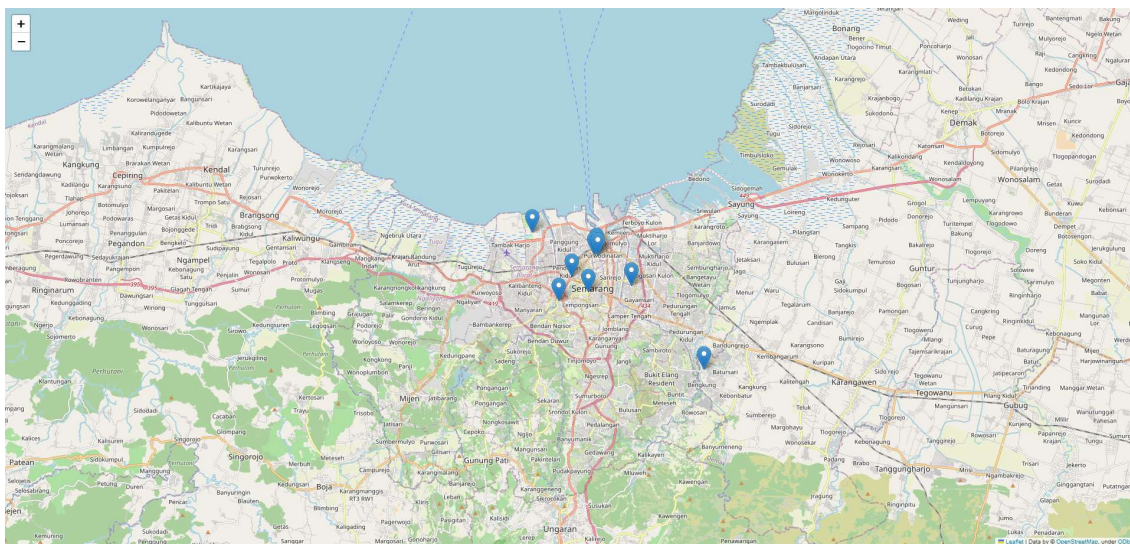
```

130:
131:     print("--- Memulai Praktikum 6: File Handling Tambahan (Log) ---")
132:
133:     # Hapus log lama jika ada (opsional, untuk memulai log bersih setiap
run)
134:     # import os
135:     # if os.path.exists(FILE_LOG):
136:     #     os.remove(FILE_LOG)
137:     #     print(f"File log lama '{FILE_LOG}' dihapus.")
138:
139:     # 1. Baca data CSV
140:     df_lokasi = baca_data_lokasi(NAMA_FILE_CSV)
141:
142:     # 2. Buat list objek dari DataFrame
143:     list_semua_lokasi = buat_objek_lokasi_dari_df(df_lokasi)
144:
145:     # 3. Buat peta (yang sekarang juga menulis log)
146:     buat_peta_lokasi_folium(list_semua_lokasi, NAMA_FILE_PETA)
147:
148:     # 4. (Opsional) Jalankan lagi untuk melihat log bertambah
149:     print("\nMenjalankan pembuatan peta lagi untuk demo log append...")
150:     buat_peta_lokasi_folium(list_semua_lokasi, "peta_kedua.html") # Nama
file berbeda
151:
152:     print(f"\nSilakan periksa isi file log '{FILE_LOG}' untuk melihat
catatan proses.")
153:     print("\n--- Praktikum 6 Selesai ---")

```

Observasi:

- Fungsi baru `tulis_log` dibuat untuk menangani penulisan ke file log. Ia menggunakan `with open(..., 'a')` untuk memastikan file ditutup otomatis dan konten baru ditambahkan di akhir file (append).
- Fungsi `buat_peta_lokasi_folium` sekarang memanggil `tulis_log` di beberapa titik: saat memulai, saat berhasil menyimpan peta, dan saat terjadi error saat menyimpan.
- Jalankan skrip ini beberapa kali. Setiap kali dijalankan, baris baru akan ditambahkan ke file `proses_peta.log` (jika file tersebut sudah ada), mencatat waktu dan pesan kejadian. Ini menunjukkan cara kerja mode `'a'`.



```

jobsheet_08.ipynb U • lokasi_semarang.csv U jobsheet_11.ipynb U • proses_peta.log X
proses_peta.log
1 [2025-04-25 11:32:55] [buat_peta_lokasi_folium] Memulai pembuatan peta 'peta_interaktif_semarang.html' dengan 10 lokasi.
2 [2025-04-25 11:32:55] [buat_peta_lokasi_folium] Peta 'peta_interaktif_semarang.html' berhasil dibuat dengan 10 marker.
3 [2025-04-25 11:32:55] [buat_peta_lokasi_folium] Memulai pembuatan peta 'peta_kedua.html' dengan 10 lokasi.
4 [2025-04-25 11:32:55] [buat_peta_lokasi_folium] Peta 'peta_kedua.html' berhasil dibuat dengan 10 marker.
5

```

E. Hasil Praktikum

Lengkapi hasil tabel praktikum berikut:

No.	Nama Praktikum	Hasil Praktikum
1	Praktikum 1: Persiapan Data (CSV)	
2	Praktikum 2: Membaca CSV dengan Pandas	
3	Praktikum 3: Mendesain Kelas OOP	
4	Praktikum 4: Membuat Objek dari Data	
5	Praktikum 5: Visualisasi dengan Folium	
6	Praktikum 6: File Handling Tambahan (Log)	

F. Penugasan

1. Kumpulkan Laporan Praktikum dari jobsheet ini dalam bentuk Microsoft word sesuai dengan format jobsheet praktikum dan dikumpulkan di web LMS. (JANGAN DALAM BENTUK PDF)
2. Kumpulkan luaran kode praktikum dalam bentuk ipynb yang sudah diunggah pada akun github masing-masing. Lampirkan tautan github yang sudah di unggah melalui laman LMS.
3. **Tambahkan kode Program** mini project SIG dari bagian praktikum.

Langkah-langkah:

a) Tambah Data & Kelas:

- o Tambahkan minimal 3 lokasi baru ke file `lokasi_semarang.csv` Anda, usahakan mencakup tipe lokasi baru (misalnya, "Kantor Pemerintahan", "Museum", "Taman Kota", dll.).
- o Buat minimal satu **kelas anak baru** yang mewarisi dari `Lokasi` untuk merepresentasikan tipe lokasi baru yang Anda tambahkan (misalnya, Kantor, Museum). Implementasikan metode `get_info_popup()` yang sesuai untuk kelas baru ini.
- o Modifikasi fungsi `buat_objek_lokasi` (dari Praktikum 4) agar dapat mengenali tipe baru ini dan membuat objek dari kelas baru yang Anda definisikan.

b) Kustomisasi Marker Folium:

- o Modifikasi fungsi `buat_peta_lokasi` (dari Praktikum 5).
- o Di dalam loop yang menambahkan marker, gunakan `isinstance()` untuk memeriksa tipe objek `lok`.
- o Berdasarkan tipe objek (`TempatWisata`, `Kuliner`, `Kantor`, `Museum`, dll.), gunakan **ikon (`folium.Icon`) atau warna marker yang berbeda**. Misalnya, tempat wisata berwarna biru, kuliner berwarna merah, kantor berwarna abu-abu. (Lihat dokumentasi `folium.Marker` dan `folium.Icon` untuk opsi kustomisasi).

c) Baca Konfigurasi Peta dari File:

- o Buat file teks sederhana baru, misalnya `config_peta.txt`.

- Isi file ini dengan koordinat latitude, longitude, dan level zoom awal untuk peta, masing-masing di baris terpisah. Contoh isi `config_peta.txt`:
 - `-6.9929`
 - `110.4200`
 - `13`
 - Modifikasi fungsi `buat_peta_lokasi`:
 - Di awal fungsi, **baca** file `config_peta.txt` menggunakan file handling (`with open(...), mode 'r'`).
 - Gunakan `try...except` (misalnya `ValueError` saat konversi ke float/int, `IndexError` jika baris kurang) untuk menangani kemungkinan error saat membaca file konfigurasi. Jika error, gunakan nilai default (misalnya, koordinat Semarang).
 - Gunakan nilai yang dibaca dari file ini untuk parameter `location` dan `zoom_start` saat membuat objek `folium.Map`.
- d) **Kode Utama:**
- Pastikan kode utama Anda memanggil fungsi-fungsi yang sudah dimodifikasi untuk membaca data, membuat objek (termasuk tipe baru), dan membuat peta dengan marker yang dikustomisasi berdasarkan konfigurasi file.
 - Pastikan peta HTML yang dihasilkan (`.html`) menunjukkan marker dengan ikon/warna berbeda sesuai tipe lokasi.

G. Kesimpulan

Jobsheet ini mendemonstrasikan kekuatan penggabungan prinsip-prinsip Pemrograman Berorientasi Objek (OOP) dengan library Python eksternal yang populer untuk tugas-tugas spesifik seperti manipulasi data dan visualisasi. Dengan merancang kelas-kelas yang merepresentasikan entitas dunia nyata (lokasi geografis) dan memanfaatkan inheritance serta polimorfisme, kita dapat membuat struktur data yang logis dan mudah dikelola. Integrasi dengan Pandas memudahkan pembacaan dan pemrosesan data dari sumber eksternal seperti CSV, sementara Folium memungkinkan visualisasi data spasial tersebut ke dalam peta interaktif yang menarik. Kemampuan untuk membaca dan menulis file (file handling) melengkapi toolkit ini, memungkinkan penyimpanan log atau pembacaan konfigurasi. Secara keseluruhan, jobsheet ini menunjukkan bagaimana konsep OOP menjadi fondasi yang kuat untuk membangun aplikasi yang lebih kompleks dan terstruktur, bahkan ketika berinteraksi dengan berbagai library dan format data.

H. Daftar Pustaka

1. Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
2. Guttag, J. V. (2016). Introduction to Computation and Programming Using Python (With Application to Understanding Data, 2nd ed.). MIT Press.
3. Python Software Foundation. Python 3 Documentation. Diakses dari <https://docs.python.org/3/>
4. Python Software Foundation. The Python Tutorial - Classes. Diakses dari <https://docs.python.org/3/tutorial/classes>.
5. Python Software Foundation. Built-in Exceptions. Diakses dari <https://docs.python.org/3/library/exceptions.html>
6. Python Software Foundation. abc — Abstract Base Classes. Diakses dari <https://docs.python.org/3/library/abc.html>
7. Das, B. N. (2018). Learn Object-Oriented Programming in Python. Packt Publishing.